



Raquette connectée, geste sportif, musicalité

Contexte

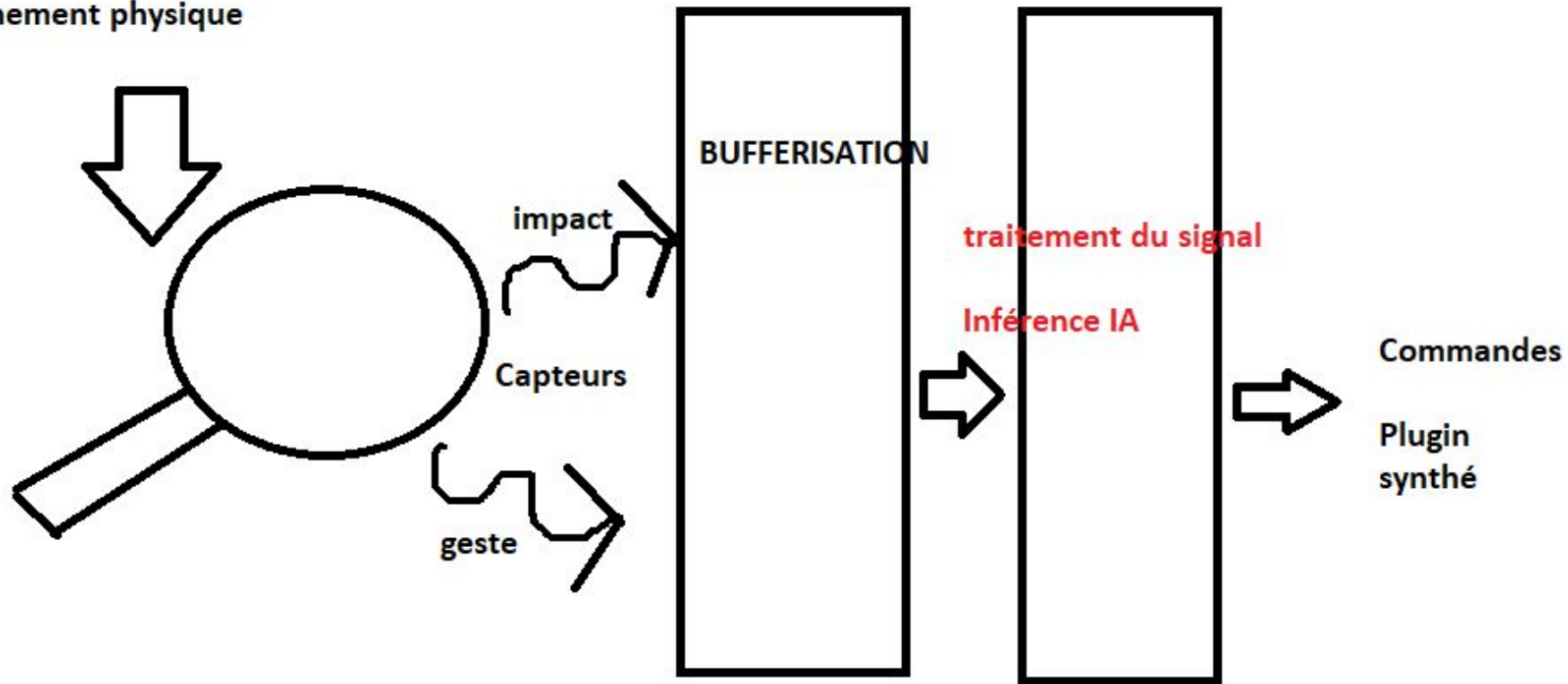
NIME, retirer de l'information pour la musique dans les gestes à l'aide des (nouvelles) technologies de capteurs.

Réaliser un traitement du signal pour décortiquer la physique du geste.

Mais surtout réaliser un encodage (à l'aide d'un CNN/réseau de neurones par exemple) de ces informations (nombreuses) pour obtenir une catégorisation de geste.

Dernier prototype

Evènement physique



La raquette

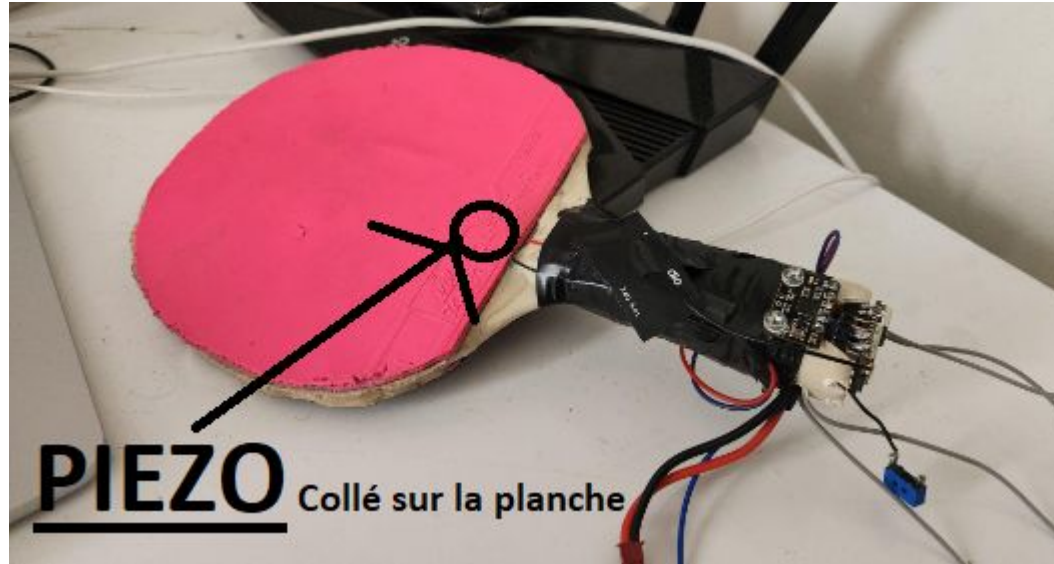
La raquette est en plastique. J'y loge la carte électronique pour le traitement et envois des données, dans le manche avec une batterie.

La captations des informations se fait selon deux axes (choix découlants de deux premiers prototypes):

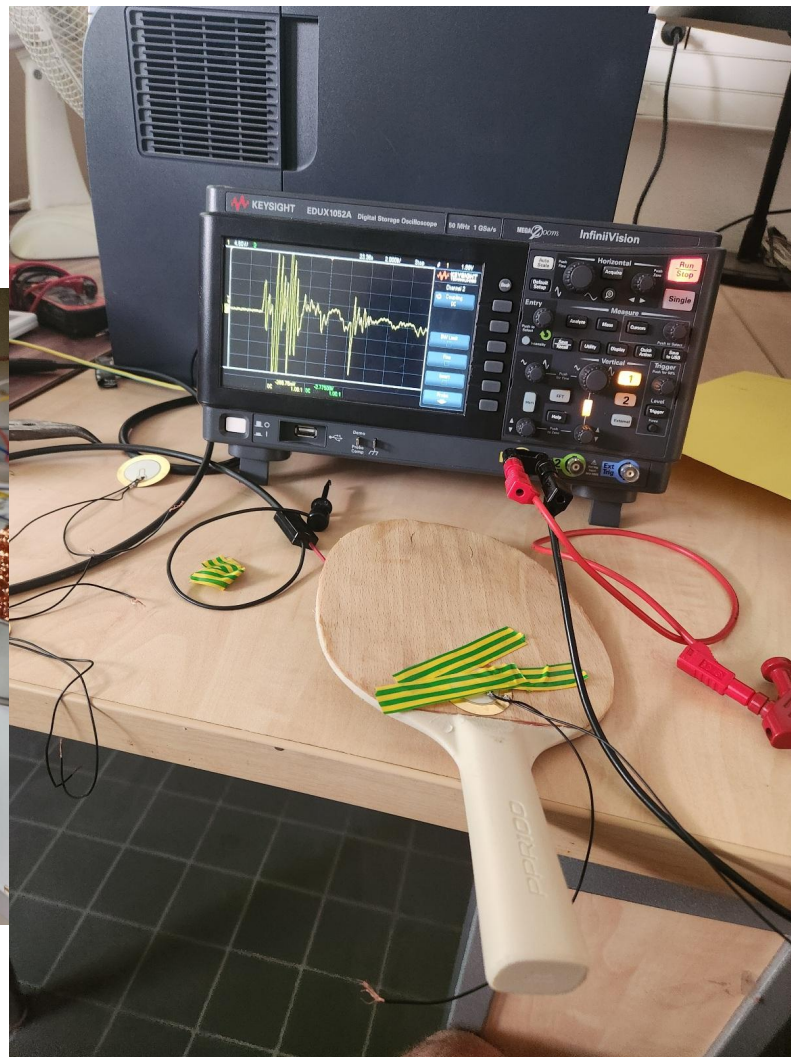
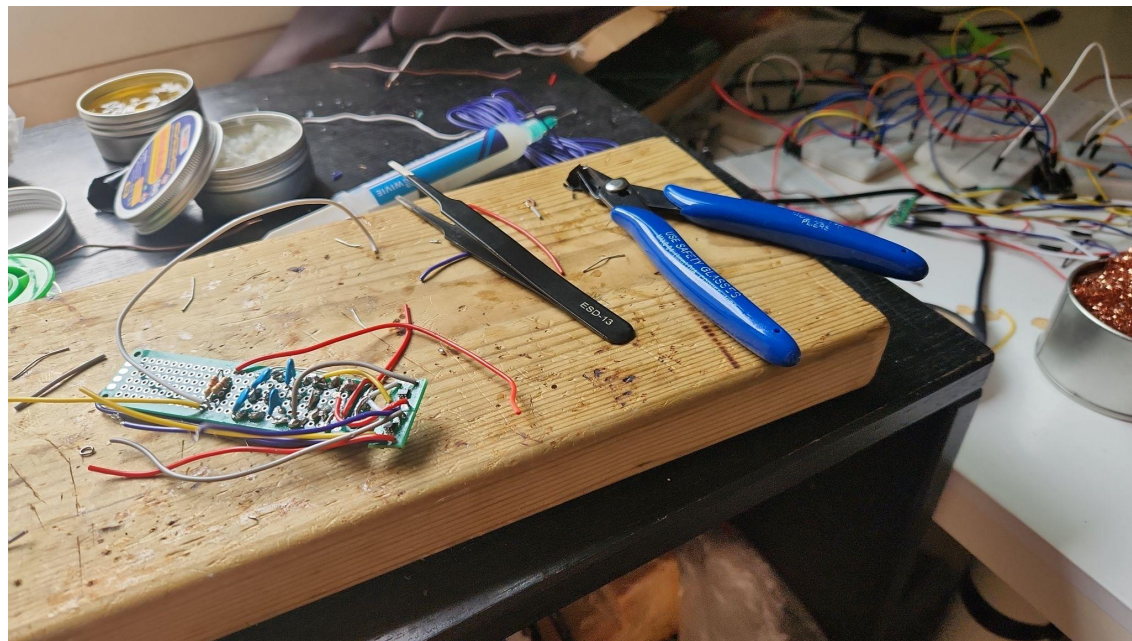
- capteur piezo, pour l'impact
- capteur imu 6dof, pour avoir de l'information sur le geste

Impact

Dispositif :

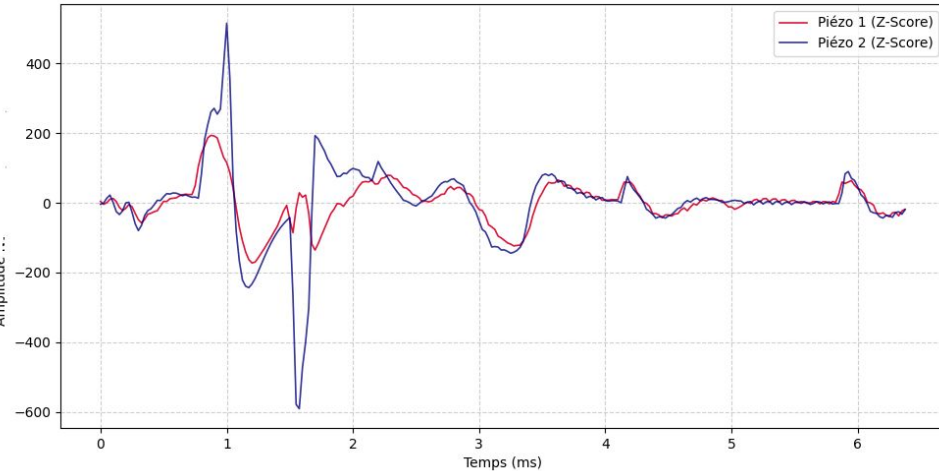


Il y a un piézo sur chaque face, non gênants pour le jeu, près du manche. Chaque piezo se tord lorsque qu'une onde de flexion anime la palette. Le collage est fait avec une très fine couche de colle époxy. Ensuite les bords sont noyés dans l'époxy.

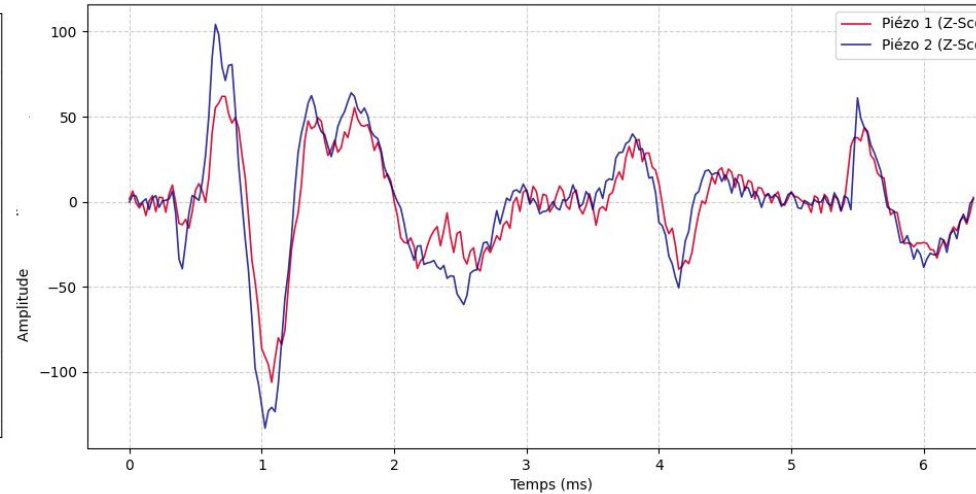


Mesures des deux signaux piézos à l'instant de l'impact

Fichier [21/40] : donnees_0001.pt | Classe: COTE2
Clic gauche / Flèche droite = Suivant | Clic droit / Flèche gauche = Précédent



Fichier [16/40] : donnees_0015.pt | Classe: COTE1
Clic gauche / Flèche droite = Suivant | Clic droit / Flèche gauche = Précédent



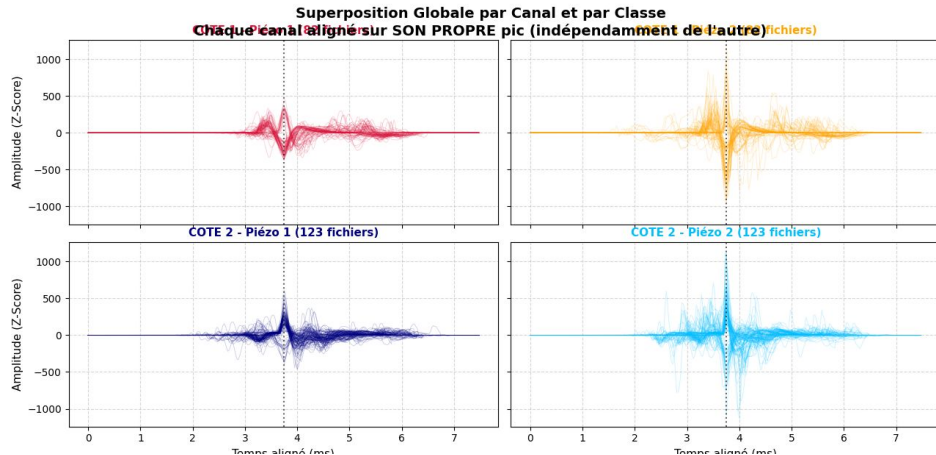
On voit bien la physique de l'impact et de la vibration qui le suit. Impact fort à gauche, faible à droite. Dans l'ordre on lit : petit pic compression revêtement, pic grosse compression, et dernier pic de rebond, puis vibrations. Beaucoup d'informations sur cet impact. Force, revêtement, et aussi le côté de l'impact. Aussi la forme de l'enregistrement est toujours le même (sur des centaines d'enregistrements, cela est visible). Le signal est exactement la force exercée par l'onde de flexion (conditionnement du signal à l'aide d'un amplificateur de charge)

CNN1D reconnaissance côté VS Algorithme

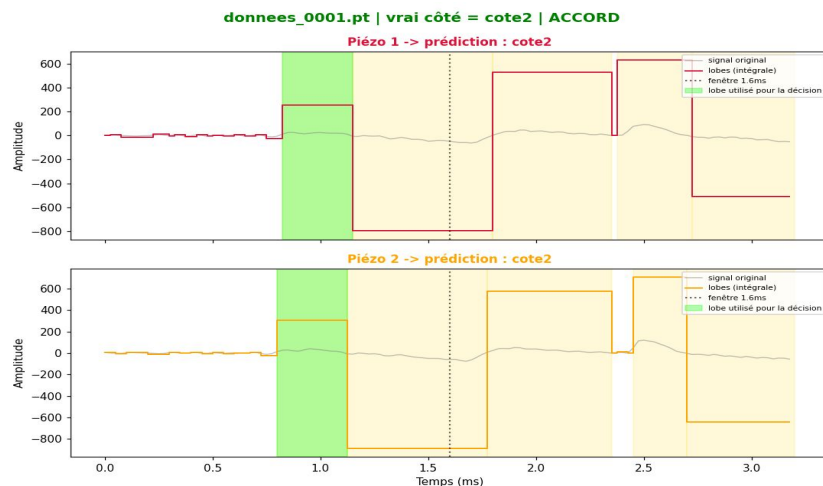
Pourquoi reconnaître le côté? Pour l'amorti, information côté + accélération perpendiculaire sens opposée => détection amorti facile pour le machine learning.

Ensuite c'est un test pour voir si les données se prêtent bien à un cnn1d pour chercher d'éventuelles autres informations (rotation, zone d'impact ou autre).

J'ai mis en place architecture cnn via pytorch. Ensuite j'ai converti le modèle pour qu'il rentre sur ma carte électronique esp32-s3 qui utilise instructions SIMD, plus rapides. L'utilisation d'un cnn pour avoir une mesure de quel côté la balle a frappé. Les résultats sont bons. f1score de 0.95 train loss et val loss de 0.2 environs. Testé directement via modèle implémenté carte et directement sur python avec la réception des données en wifi, les résultats sont mauvais. Le modèle généralise mal, les données sont de mauvaise qualité (mauvais centrage) et peu nombreuses. CF SUPERPOSITION des impacts selon les différents piezos et côtés d'impacts en centrant sur le max:



J'ai ainsi fabriqué un algorithme de détection de côté qui traite les données, car visuellement quelque chose se répète (l'aire sous la courbe x largeur segment positif ou négatif):



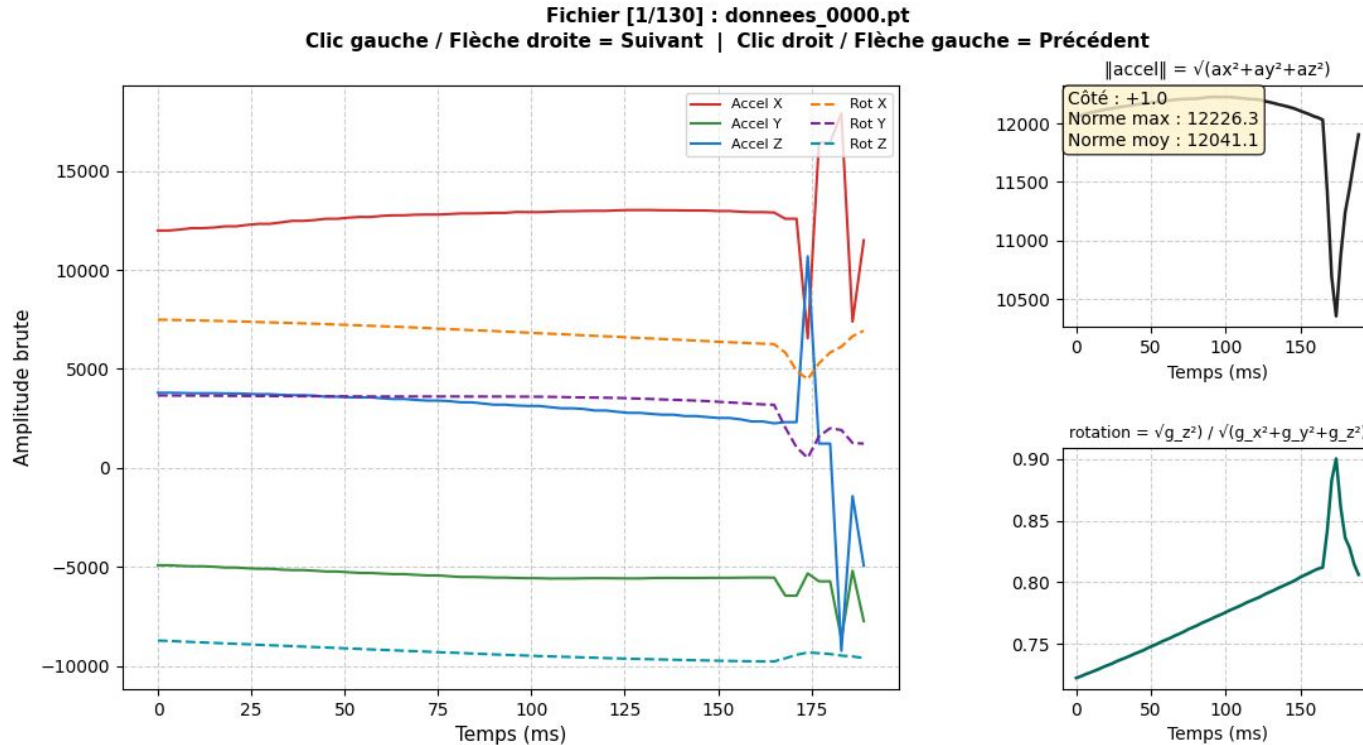
Cet algorithme offre un f1 score de 0.92 et plus et fonctionne très bien en test réel. J'avais essayé findpeak() sur python pour détecter les pics et le sens des pics, mais c'est très mauvais. (f1 score de 0.70/0.80).

Mesures accéléromètre 6 dof MPU 6050 sur 192 ms

Geste

acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z, en int16_t

Transformée en float avec normalisation



Encoder les données de gestes

Features pour distinguer (encoder) mes mesures.

Puisque que les données ressemblent à des droites les features suivantes (suffisantes pour décrire complètement l'information) sont choisies :

- moyenne
- Pente grossière
- Pente
- min
- max

RESULTATS

Des résultats intéressants lorsque l'on fait un traitement statistique des enregistrements d'accéléromètre Topspin/Tape :

- la rotation n'est pas caractérisée physiquement par l'accélération dans le plan de la raquette (statistiques) ⇒ tout est dans l'impact, et le mouvement est en totalité dans ce plan de raquette, indépendamment de geste "topspin" ou "tapé", le joueur ne frappe pas frontalement (ou peu)
- Les features permettent de très bien distinguer topspin/tape, coup droit/revers aussi, bon encodage → le modèle neural network simple pourra très bien lire/traiter les données. Le seul problème étant que les données se séparent trop bien avec les features et les sons varient peu ; c'est dichotomique.

RESULTAT en situation réelle

Je commande plugin VITAL avec une catégorisation topspin/tape. J'ai le son de la rotation.

Cela fonctionne très bien, le modèle fonctionne. Je vais retourner au club pour tester enfin la raquette comme outils, et je pourrai le régler et l'adapter au millimètre selon les besoins. On peut maintenant étudier le côté musicale.

CF vidéo démonstration en club.

Pour la prochaine version j'ai encore de la place pour faire tous les calculs d'inférence IA dans la carte électronique, et je pourrais envoyer les données en bluetooth à un téléphone, avec une latence raisonnable, pour activer un plugin simple, léger et rigolo, permettant un utilisation extrêmement facile de la raquette.

REAPER, Plugin VITAL

pingpong - REAPER v7.45 - Registered to Nicolas Scheffler (Licensed for personal/small business use)

File Edit View Insert Item Track Options Actions Help

[audio device closed]

The screenshot displays the REAPER v7.45 interface. The top menu bar includes File, Edit, View, Insert, Item, Track, Options, Actions, and Help. The main workspace is divided into several panels:

- Left Panel:** Master track controls showing -9.09dB, M, S, ROUTE, and FX buttons. Below it, a track control for track 1 with a TRIM button and a volume knob.
- Center Panel:** A JS Development Environment window titled "JS Development Environment". It shows a script for "d clenche note Midi" with parameters for Note, Velocity, Trigger, notetime, and gain (dB). The script is as follows:

```
hesc:d clenche note Midi

slider1:60<0,127,1>Note
slider2:100<1,127,1>Velocity
slider3:0<0,1,1>Trigger
slider4:0<0,5000,1>notetime
slider5:gain_db=0<-150,12,1>gain (dB)
```
- Right Panel:** A table of system variables and their values. The table has columns for Name, Value, and R... (Rate). The variables listed are: d_gain (0), last_gain (1), next_gain (1), note_time (4686), old_trigger (1), slider6 (0), beat_positi... (20), ext_gr_met... (10000), ext_midi_b... (0), ext_noden... (0), ext_noinit (0), ext_tail_size (0), gain_db (0), gfx_a (0), gfx_a2 (0), gfx_b (0), gfx_clear (0), gfx_dest (0), gfx_ext fla... (0), and qfx_ext ret... (0).
- Bottom Panel:** A VITAL plugin window titled "FX: Track 1". It shows a list of FX: JS: d clenche note Midi, VST3: Vital (Vital Audio), and JS: Delay. The VITAL plugin parameters are: Note (66.0), Velocity (60.0), Trigger (1.0), notetime (5000.0), and gain (dB) (0.0).

The bottom status bar shows the scroll timeline, the current time 6.1.00 / 0:10.000, the status [Stopped], and the rate 1.0.

Mesures topspin/tape/amorti pour réseau de neurones

